

GO DATA PROFILER

Installation, Usage, Architecture and Profiling Guide

Purpose

A practical engineering guide for deploying and operating the Go-based database profiling service.

Supported source databases

PostgreSQL | MySQL | Microsoft SQL Server

Deployment model

Docker Compose for local and portable deployments. The profiler API and metadata database are separate from the source database being profiled.

Repository

`adm-l/data_profiler`

Document principle

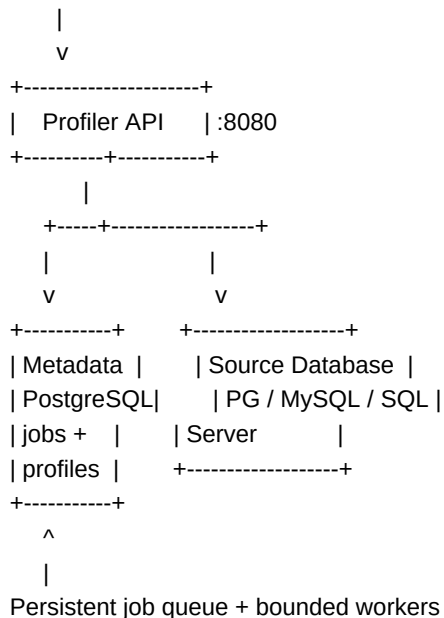
This guide describes implemented capabilities and clearly separates current behavior from recommended production hardening.

1. OVERVIEW AND ARCHITECTURE

Go Data Profiler is an asynchronous database profiling service. A client submits a job over HTTP. The service persists the job, processes it through a bounded worker pool, reads metadata and data from the source database, calculates profiling metrics, evaluates quality rules, stores a snapshot, and compares it with the previous snapshot.

ARCHITECTURE

Client / UI / curl



Boundary: source data is read from the source DB; job state and profile snapshots are stored in the profiler metadata DB.

2. WHAT IS PROFILED

Table-level

- Row count and sampling status
- Duplicate row count and percentage
- Quality score and checks
- Numeric correlations
- Foreign-key relationship discovery
- Historical snapshots and drift

Column-level

- Null, distinct, empty and whitespace counts
- Numeric min/max/average/sum/variance/stddev
- Skewness, kurtosis and percentiles
- Zero and outlier statistics; histograms
- Text length, top values, patterns and entropy
- Date ranges, future-date counts and distributions
- PII indicators using header and value evidence

Interpretation rule

Do not rely on one metric. Read completeness, cardinality, distribution, quality checks, PII indicators and drift together.

3. INSTALLATION AND CONFIGURATION

Recommended Docker setup

```
git clone https://github.com/adm-l/data_profiler.git
cd data_profiler
docker compose up --build -d
docker compose ps
docker compose logs --tail=100 profiler-api
```

Verify

```
curl http://localhost:8080/healthz
curl http://localhost:8080/readyz
curl http://localhost:8080/metrics
```

Default configuration

```
HTTP_ADDR=:8080
PROFILER_DATABASE_URL=postgres://profiler:profiler@profiler-db:5432/profiler_db?sslmode=disable
JOB_WORKERS=4
LOG_LEVEL=info
PROFILE_SOURCE_MAX_OPEN_CONNS=4
PROFILE_SOURCE_MAX_IDLE_CONNS=2
PROFILE_SOURCE_CONN_MAX_LIFETIME=30m
```

Requirements

- Docker Engine + Docker Compose
- Network access from profiler container to source DB
- Read/metadata permissions for source account
- PostgreSQL is used by Compose as the profiler metadata store

Security baseline

Use a least-privilege source account. Do not grant write access to application tables.

4. DATABASE CONNECTIONS AND API

PostgreSQL DSN

postgres://USER:PASSWORD@HOST:5432/DATABASE?sslmode=disable

MySQL DSN

USER:PASSWORD@tcp(HOST:3306)/DATABASE

SQL Server DSN

sqlserver://USER:PASSWORD@HOST:1433?database=DATABASE&encrypt=disable

Docker note

For Docker Desktop, host.docker.internal commonly reaches a database on the host. Linux environments may require explicit host-gateway/network configuration.

Create a job

```
curl --location http://localhost:8080/api/v1/profile-jobs \
--header 'Content-Type: application/json' \
--data-raw '{"source":{"type":"postgres","dsn":"postgres://postgres@host.docker.internal:5432/user_db?sslmode=disable"},"schema":"public","table":"users2","sample_size":1000}'
```

Check status

```
curl http://localhost:8080/api/v1/profile-jobs/<JOB_ID>
```

Read result

```
curl http://localhost:8080/api/v1/profile-jobs/<JOB_ID>/result
```

Other endpoints

GET /healthz

GET /readyz

GET /metrics

GET /api/v1/profiles/history?schema=public&table=users2&limit=20

GET /api/v1/profiles/drift?schema=public&table=users2

5. JOBS, QUALITY, HISTORY AND DRIFT

Idempotency

Use Idempotency-Key for safe client retries. The key is persisted and protected by a unique index.

```
--header "Idempotency-Key: users2-profile-v1"
```

If an older Docker volume lacks the column:

```
docker exec -i go-profiler-db psql -U profiler -d profiler_db <
migrations/003_idempotency.sql
```

Development-only reset:

```
docker compose down -v
docker compose up --build -d
```

Quality defaults

- Null percentage threshold: 20%
- Dominant value threshold: 80%
- High cardinality threshold: 95%
- Empty values: not allowed by default
- Whitespace-only values: not allowed by default

Exact threshold values currently pass.

History and drift

A completed profile is stored as a snapshot. Drift can report added/removed columns, type/nullability changes, meaningful null/distinct changes, outlier changes, PII classification changes, duplicate-row changes and quality-score changes.

Sampling

sample_size makes large-table profiling practical. Results explicitly indicate whether sampling occurred. Sampling is not automatically representative for every business distribution.

6. OPERATIONS, SECURITY AND ROADMAP

Performance and reliability

- Bounded job workers prevent unbounded concurrent scans.
- Source DB connections are separately bounded.
- Profiling work uses context cancellation/timeouts.
- Queued jobs persist across restart.
- Stale running jobs can be recovered.
- Correlation analysis is capped to a bounded number of numeric columns.
- Duplicate detection can be expensive on wide or very large tables.

Security

- Source DSNs are redacted from job API responses.
- Prefer secret-manager references over raw credentials.
- Use TLS where required.
- Protect the API with authentication/authorization in non-local deployments.
- Treat PII detection as an indicator, not a complete privacy classification system.

Production checklist

1. Durable/managed metadata PostgreSQL.
2. Secrets manager and least privilege.
3. API authentication and network controls.
4. TLS for API and source DB connections.
5. Monitor API latency, job duration, failures, queue depth and source load.
6. Retention policy for profile snapshots.
7. Benchmark duplicate detection on representative data.
8. Use a proper metadata migration runner.
9. Add distributed job claiming/locking before multi-instance scaling.

Recommended next improvements

- Stronger PII classification to reduce generic-name false positives.
- Configurable duplicate detection and database-native optimization.
- Scheduled profiling and snapshot retention.
- Secret references/encrypted credential handling.
- Authentication/authorization and distributed worker coordination.

Validation

```
go test ./...
go vet ./...
docker compose up --build -d
docker compose ps
```